JAIDEV EDUCATION SOCIETY'S
J D COLLEGE OF ENGINEERING AND MANAGEMENT
KATOL ROAD, NAGPUR
Website: www.jdcoem.ac.in E-mail: info@jdcoem.ac.in
(An Autonomous Institute, with NAAC "A" Grade)
Affiliated to DBATU, RTMNU & MSBTE Mumbai
Department of Artificial Intelligence
"Place to Learn, Chance to Grow"
2025-2026 (EVEN Semester)

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Branch: Artificial Intelligence **Semester: 6thSem 3rd Year**

Subject: LFTO

UNIT -II

Fundamentals of Fine-Tuning: Transfer Learning and Domain Adaptation, Full Fine-tuning vs. Feature-based Fine-tuning, Dataset preparation and preprocessing for fine-tuning, Evaluation metrics and validation techniques

---

Fundamentals of Fine-Tuning: Transfer Learning and Domain Adaptation

Fine-tuning, transfer learning, and domain adaptation are key techniques in machine learning, particularly in deep learning. These approaches allow models to leverage previously learned knowledge to solve new tasks or adapt to different domains with less data. Let's break them down:

1. Transfer Learning

Transfer learning refers to the technique of taking a pre-trained model (trained on one task) and adapting it for a new, but related, task. Instead of training a model from scratch, which requires a large amount of data and computational resources, transfer learning allows you to "transfer" knowledge gained from a source task to a target task. The key idea is that knowledge learned in one domain can be valuable for a different domain, especially when there are similarities in the underlying patterns.

Key Steps in Transfer Learning:

1. **Pre-training**: The model is initially trained on a large dataset (source task) to learn general features. For example, models like GPT, BERT, and ResNet are pre-trained on large datasets like ImageNet (images) or large corpora of text.
2. **Fine-tuning**: After pre-training, the model is adapted to the specific target task. The model's weights are slightly adjusted by training on a smaller, task-specific dataset.

Example:

- **Source Task**: Training a neural network to classify images of everyday objects (e.g., ImageNet).
- **Target Task**: Fine-tuning the model to classify medical images, like X-rays, which are not as common in ImageNet.

In this case, the model already knows how to identify basic features like edges, textures, and shapes, and fine-tuning helps it specialize for medical images.

Types of Transfer Learning:

- **Feature Extraction**: Using the pre-trained model as a fixed feature extractor. The learned features from the earlier model are passed through a new classifier.
- **Fine-Tuning**: Unfreezing the weights of some layers of the pre-trained model and re-training them on the target task with a smaller learning rate.

2. Domain Adaptation

Domain adaptation is a specific type of transfer learning where a model trained on data from one domain (source domain) is adapted to work well on a different but related domain (target domain). It’s particularly useful when you have labeled data for the source domain but very little labeled data for the target domain.

While transfer learning works well when the source and target domains are similar, domain adaptation techniques aim to handle cases where there's a large distribution mismatch between the source and target domains.

Key Concepts in Domain Adaptation:

- **Domain Shift**: This refers to the difference in data distributions between the source and target domains. For instance, you might train a model on data from sunny weather and want it to work on rainy day images, which is a domain shift.
- **Unsupervised Domain Adaptation**: In this case, there is no labeled data available for the target domain. The model is trained on labeled data from the source domain and adapts to the target domain using unlabeled data from the target domain.

- **Supervised Domain Adaptation**: Labeled data is available for both the source and target domains, but the domains are different, requiring adaptation techniques to reduce the domain gap.

Domain Adaptation Techniques:

1. **Adversarial Training**: A technique inspired by Generative Adversarial Networks (GANs). Here, a domain discriminator tries to distinguish between the source and target domains, while the feature extractor tries to fool the discriminator, resulting in domain-invariant features.
2. **Domain-Adversarial Neural Networks (DANN)**: This network consists of a feature extractor, label predictor, and a domain classifier that encourages the feature extractor to learn representations that are discriminative for the task but indistinguishable between domains.
3. **Self-ensembling**: This involves creating two models—one trained on the source domain and one on the target—and ensuring their predictions align over time. This helps the model become more robust to domain changes.
4. **Data Augmentation**: Augmenting the target domain data (e.g., rotating images, changing colors) to make it more similar to the source domain and reduce the gap between domains.

Example of Domain Adaptation:

- **Source Domain**: A facial recognition model trained on well-lit, frontal images of people.
- **Target Domain**: The same model needs to work on facial images taken in low-light conditions or at different angles. Domain adaptation will help the model adjust to these new conditions.

Fine-Tuning vs. Domain Adaptation

- **Fine-tuning** is generally about adapting a pre-trained model to a new task, with a focus on leveraging learned representations for a new classification or regression task.
- **Domain Adaptation** specifically focuses on adapting a model to perform well in a new domain that has a different distribution, even if the task itself remains similar.

Why Fine-Tuning and Domain Adaptation Matter

- **Reduced Data Requirements**: Fine-tuning and domain adaptation reduce the need for vast amounts of labeled data, making them useful in scenarios where labeling data is costly or time-consuming.
- **Faster Convergence**: By using pre-trained models and adjusting them slightly for new tasks, models can converge much faster than if they were trained from scratch.
- **Better Generalization**: Transfer learning can lead to better generalization, especially when the amount of data in the target domain is small but similar in nature to the source domain.

Challenges:

- **Negative Transfer**: Sometimes, the knowledge transferred from the source domain may not help the target domain, and it could even harm performance. This is called negative transfer.
- **Domain Gap**: If the source and target domains are too different, the performance of the adapted model may suffer, and special techniques are needed to address this gap.

Example Use Cases:

- **Natural Language Processing (NLP)**: A model pre-trained on a large corpus (like BERT) can be fine-tuned on a specific task, such as sentiment analysis, named entity recognition (NER), or question answering.
- **Computer Vision**: Pre-trained models like ResNet, VGG, and Inception can be fine-tuned for tasks like medical image classification or object detection in images from a different domain (e.g., outdoor scenes vs. indoor scenes).

Conclusion

Fine-tuning, transfer learning, and domain adaptation are crucial in making deep learning models more efficient and effective, especially when working with limited data for a new task or domain. These approaches help leverage existing knowledge from one domain and transfer it to another, reducing the need for extensive datasets and improving the ability of models to generalize. The key difference between them is that while transfer learning focuses on adapting to a new task, domain adaptation specifically deals with handling domain shifts to ensure better performance in a new environment.

Dataset Preparation and Preprocessing for Fine-Tuning

1. Introduction to Fine-Tuning

Fine-tuning is a process in machine learning where a pre-trained model is further trained on a specific task or domain dataset. Pre-trained models already learn general patterns from large datasets, and fine-tuning helps adapt this knowledge to a specialized task. This approach saves training time, reduces computational cost, and improves accuracy even with limited data. Fine-tuning is widely used in NLP, computer vision, and speech recognition.

2. Dataset Preparation

Dataset preparation involves collecting, organizing, labeling, and structuring data so it can be effectively used for fine-tuning a model.

2.1 Data Collection

Data collection is the first and most important step. It involves gathering raw data relevant to the task. The data should represent real-world scenarios and include enough diversity to avoid bias. Common sources include public datasets, APIs, web scraping, sensors, and internal organizational data. Ethical considerations such as data privacy, consent, and copyright must be followed.

2.2 Data Annotation / Labeling

Annotation is the process of assigning correct labels to raw data. Supervised learning models require labeled data. For text classification, labels could be sentiments; for images, labels could be object names or bounding boxes. High-quality labeling ensures better model learning. Poor labeling introduces noise and reduces performance. Annotation can be manual, semi-automatic, or crowdsourced.

2.3 Dataset Splitting

The dataset is divided into training, validation, and test sets. The training set is used to learn model parameters. The validation set helps tune hyperparameters and avoid overfitting. The test set evaluates the final performance. Proper splitting prevents data leakage and ensures fair evaluation.

2.4 Data Balancing

Data imbalance occurs when some classes have significantly more samples than others. This causes biased predictions where the model favors majority classes. Techniques such as oversampling, undersampling, SMOTE, and class weighting are used to handle imbalance. Balanced datasets improve fairness and recall for minority classes.

3. Data Preprocessing

Data preprocessing converts raw data into a clean and structured format suitable for model training. It improves learning efficiency and model performance.

3.1 Data Cleaning

Data cleaning removes incorrect, incomplete, or duplicate data. Missing values can be handled using deletion or imputation methods. Outliers and noisy samples are detected and corrected or removed. Clean data ensures accurate model learning.

3.2 Text Preprocessing

Text preprocessing standardizes textual data. It includes lowercasing, removing punctuation and special characters, tokenization, stop-word removal, stemming or lemmatization, and padding or truncation. These steps help models understand language patterns effectively.

3.3 Image Preprocessing

Image preprocessing ensures uniformity in image data. Images are resized, normalized, and converted into appropriate color formats. Data augmentation techniques such as rotation, flipping, and cropping increase dataset diversity and reduce overfitting.

3.4 Feature Engineering

Feature engineering transforms raw data into meaningful numerical features. It includes encoding categorical variables, scaling numerical features, and generating embeddings. Well-designed features improve model accuracy and convergence speed.

3.5 Tokenization and Encoding

Tokenization breaks text into words or sub-words. Encoding converts tokens into numerical IDs understood by models. Modern models use subword tokenization techniques like BPE or WordPiece to handle unknown words.

4. Data Formatting for Fine-Tuning

Fine-tuning requires data to be formatted according to model specifications. Common formats include CSV, JSON, and TFRecord. Each data sample must align input-output pairs correctly to avoid training errors.

5. Quality Checks

Quality checks ensure data reliability. This includes verifying labels, checking consistency, removing noise, and validating distributions. Quality assurance prevents poor model performance.

6. Importance of Proper Preprocessing

Proper preprocessing improves model accuracy, generalization, and stability. It reduces training time, prevents overfitting, and enhances robustness in real-world applications.

7. Conclusion

Dataset preparation and preprocessing are the foundation of fine-tuning. A well-prepared dataset leads to reliable, accurate, and efficient machine learning models, while poor data quality results in weak and biased systems.

Evaluation Metrics and Validation Techniques

1. Introduction

After training a machine learning or deep learning model, it is essential to evaluate its performance. Training accuracy alone is not sufficient because a model may perform well on training data but poorly on unseen data, a problem known as overfitting. Evaluation metrics measure model performance numerically, while validation techniques ensure that the model generalizes well.

2. Evaluation Metrics

Evaluation metrics are quantitative measures used to assess the quality and effectiveness of a trained model. Different machine learning tasks require different evaluation metrics such as classification, regression, and NLP metrics.

3. Evaluation Metrics for Classification Problems

3.1 Confusion Matrix

A confusion matrix is a table that describes the performance of a classification model by comparing actual and predicted values.
True Positive (TP): Correctly predicted positive samples.
True Negative (TN): Correctly predicted negative samples.
False Positive (FP): Incorrectly predicted positive samples.
False Negative (FN): Incorrectly predicted negative samples.
The confusion matrix forms the basis for many other evaluation metrics.

3.2 Accuracy

Accuracy measures the overall correctness of the model.
Accuracy = (TP + TN) / (TP + TN + FP + FN)
It is easy to understand but misleading for imbalanced datasets.

3.3 Precision

Precision measures how many predicted positives are actually positive.
Precision = TP / (TP + FP)
It is important when false positives are costly, such as in spam detection.

3.4 Recall (Sensitivity)

Recall measures how many actual positives are correctly identified.
Recall = TP / (TP + FN)
It is important when false negatives are dangerous, such as in disease detection.

3.5 F1-Score

F1-score is the harmonic mean of precision and recall.

F1 = 2 × (Precision × Recall) / (Precision + Recall)

It balances precision and recall and is useful for imbalanced datasets.

3.6 Specificity

Specificity measures how well the model identifies negative samples.

Specificity = TN / (TN + FP)

3.7 ROC Curve and AUC

The ROC curve plots the True Positive Rate against the False Positive Rate. AUC measures the model's ability to distinguish between classes. A higher AUC indicates better model performance.

4. Evaluation Metrics for Regression Problems

4.1 Mean Absolute Error (MAE)

MAE measures the average absolute difference between predicted and actual values. It is easy to interpret and less sensitive to outliers.

4.2 Mean Squared Error (MSE)

MSE measures the average squared difference between predicted and actual values. It penalizes large errors more but is sensitive to outliers.

4.3 Root Mean Squared Error (RMSE)

RMSE is the square root of MSE and penalizes large errors more heavily.

4.4 R-Squared (R^2)

R-squared measures how well the model explains variance in the data. Its value ranges from 0 to 1.

5. Evaluation Metrics for NLP Tasks

5.1 BLEU Score

BLEU score is used in machine translation to measure similarity between predicted and reference sentences.

5.2 Perplexity

Perplexity measures how well a language model predicts text. Lower perplexity indicates better performance.

6. Validation Techniques

Validation techniques are used to estimate how well the model performs on unseen data and to avoid overfitting.

7. Hold-Out Validation

The dataset is split into training and testing sets. It is simple and fast but less reliable for small datasets.

8. K-Fold Cross-Validation

The dataset is divided into K equal parts. The model is trained K times, each time using a different fold for validation. It provides a better performance estimate and reduces bias.

9. Stratified K-Fold Validation

This method preserves class distribution in each fold and is best for imbalanced datasets.

10. Leave-One-Out Cross-Validation (LOOCV)

Each data sample is used once as test data. It is very accurate but computationally expensive.

11. Time-Series Validation

Used for sequential data where training is done on past data and testing on future data to prevent data leakage.

12. Validation Set Approach

A separate validation set is used for hyperparameter tuning and prevents overfitting.

13. Importance of Validation Techniques

Validation techniques help detect overfitting and underfitting, improve generalization, and assist in selecting the best model and hyperparameters.